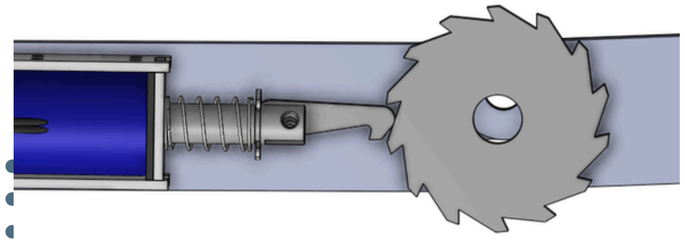
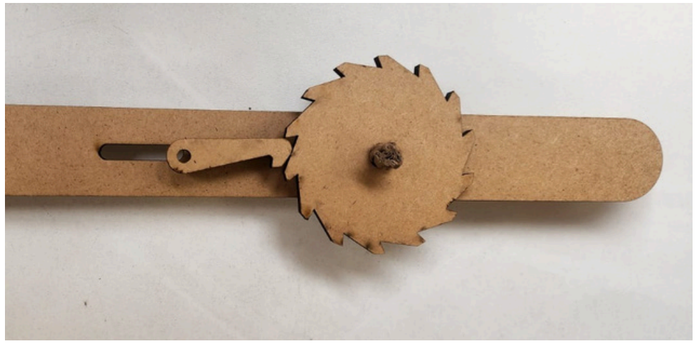
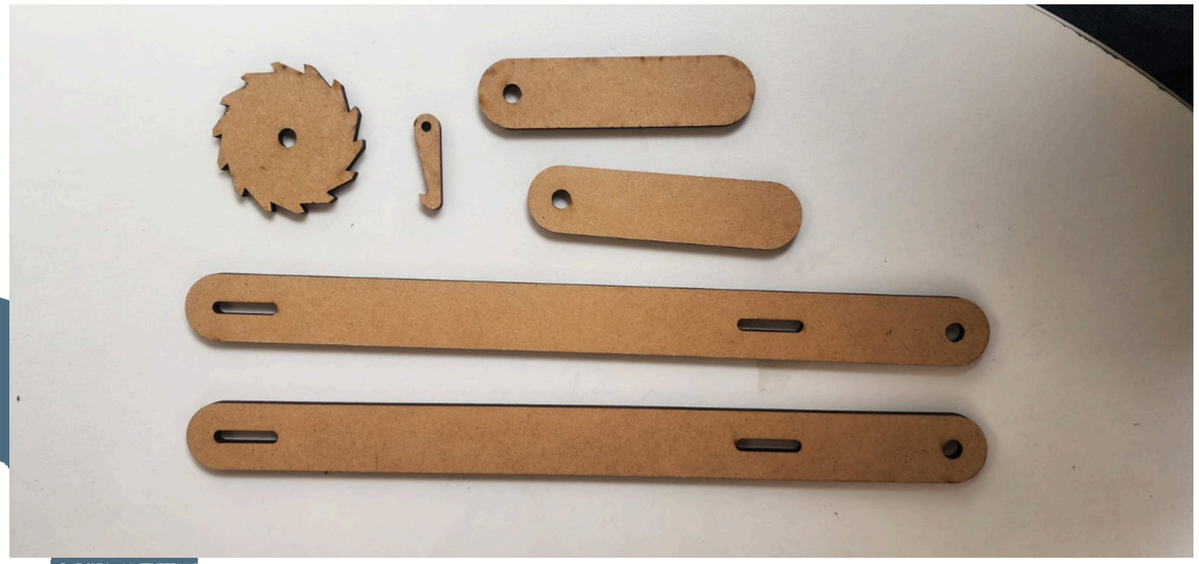
ENTREGABLE N°10

GRUPO 11

Mecánica y fabricación







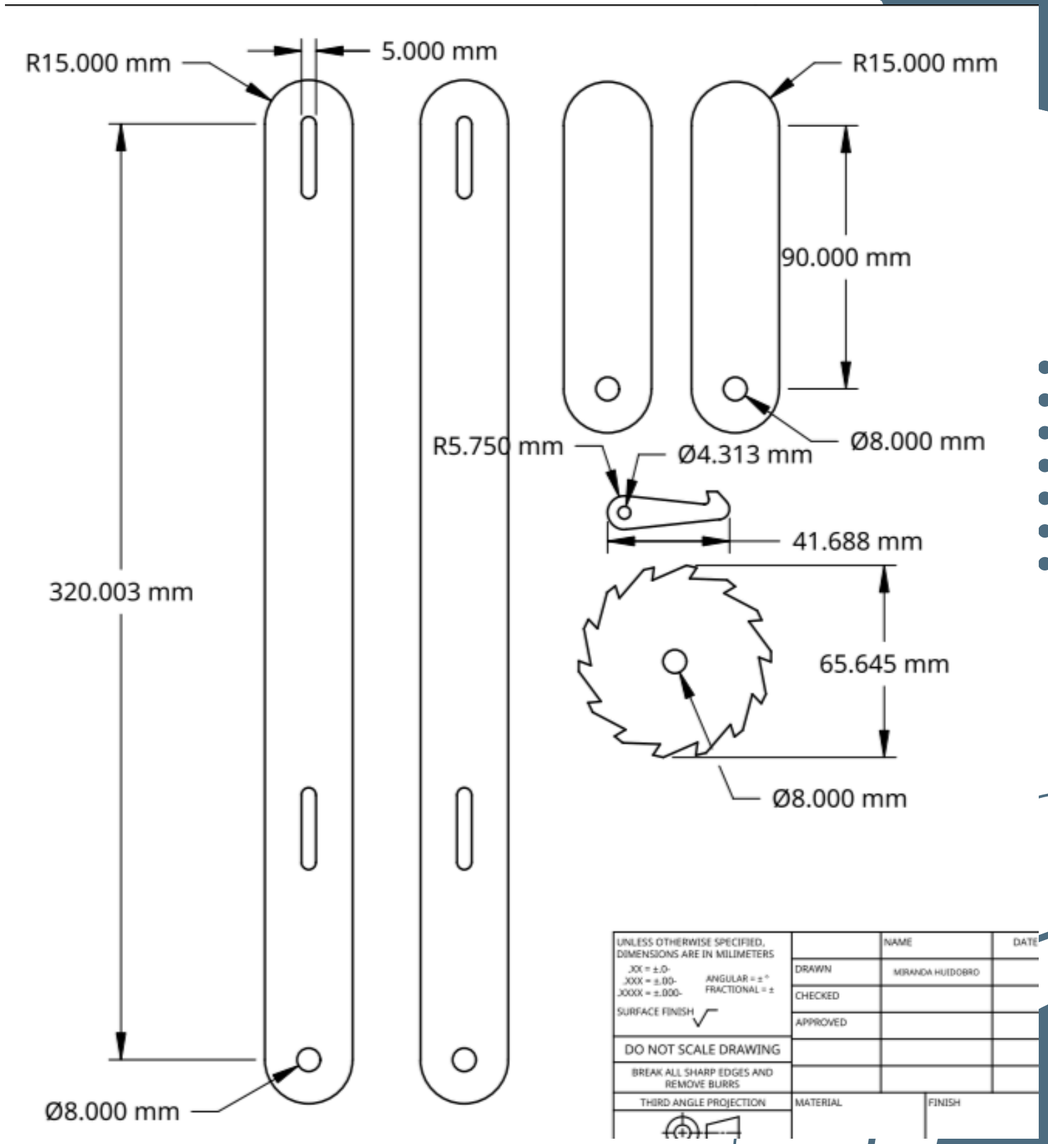
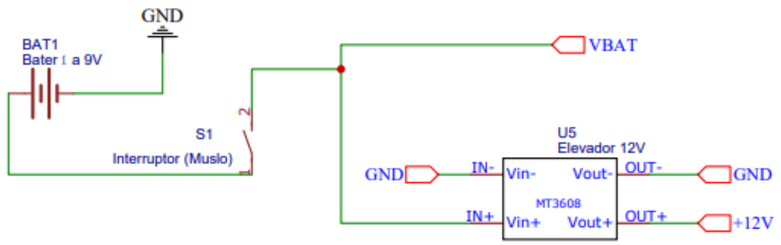
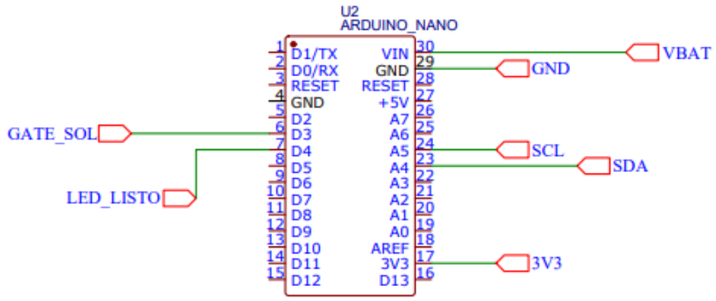


Diagrama de circuitos

Modulo de Alimentacion



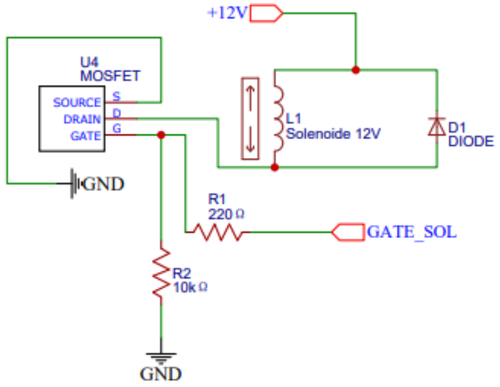
Modulo de Microcontrolador



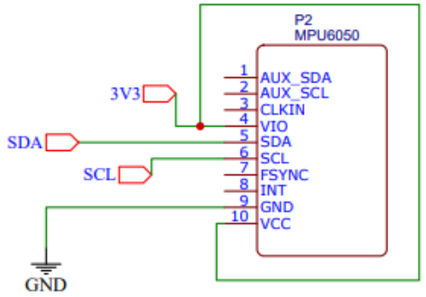
Modulo de Indicador de Arranque



Modulo del Actuador

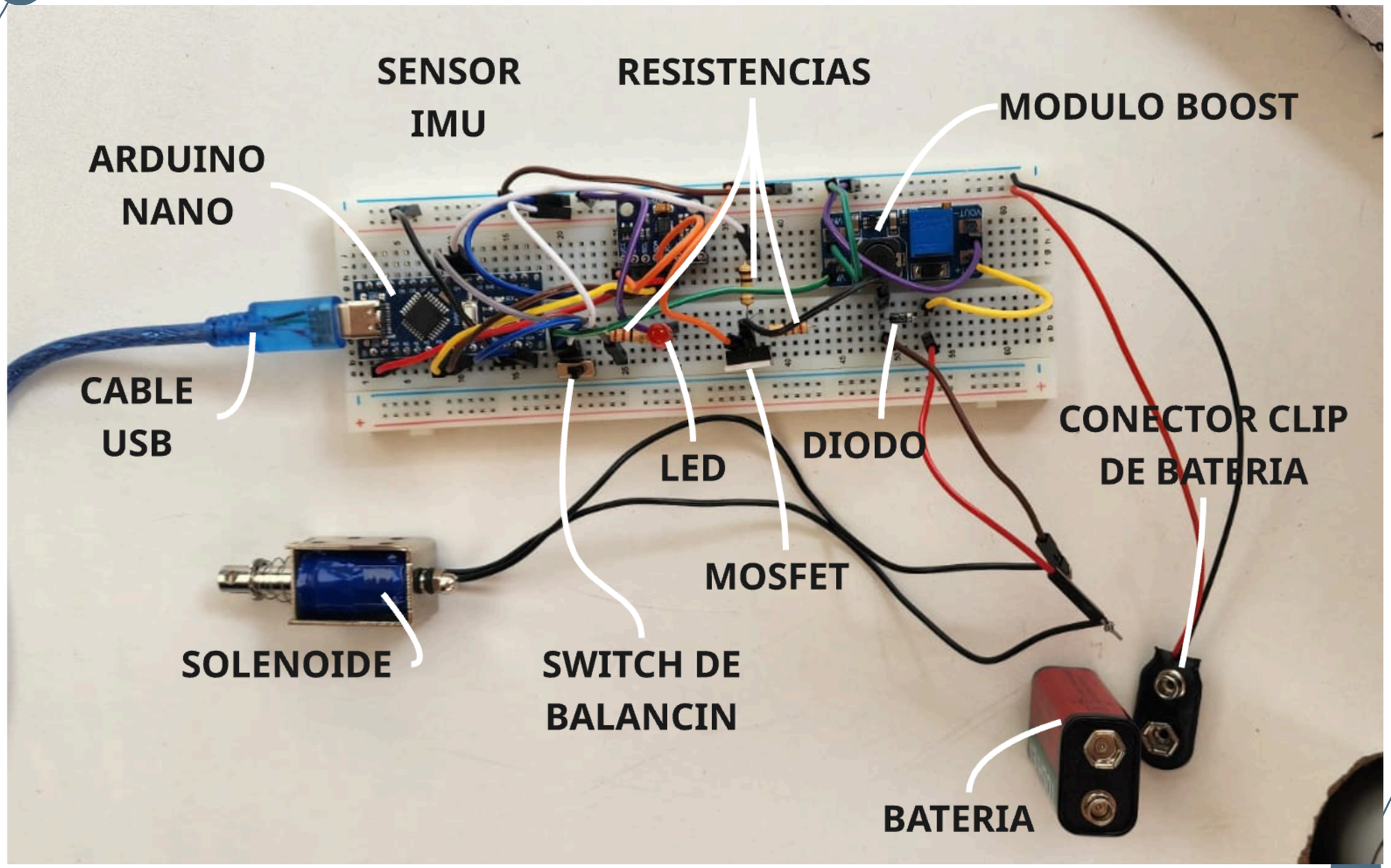


Modulo del sensor inercial



TITLE: Modulo de bloqueo de rodilla KAFO		REV: 1
UNIVERSIDAD PERUANA CAYETANO HEREDIA	Company: UPCH	Sheet:
	Date: 2026-11-02 Drawn By: Equipo 11	

Electrónica y sensores



Programación y control

Configuración global del sistema

```ino
#include <Wire.h>

// ---------- Pines ----------
const uint8_t PIN_SOLENOIDE = 3;
const uint8_t PIN_LED       = 4;

// ---------- MPU6050 ----------
const uint8_t MPU_DIR = 0x68;   // dirección I2C (AD0 sin conectar o a GND)

// ---------- Parámetros ajustables ----------
const float ANGULO_LIBERAR    = 20.0;   // grados de elevación para liberar
const float ANGULO_REENGANCHE = 8.0;    // grados para considerar "pierna recta"
const unsigned long T_CONFIRMAR_LIBERAR    = 100;   // ms sostenido para liberar
const unsigned long T_CONFIRMAR_REENGANCHE = 150;   // ms sostenido para reenganchar
const unsigned long T_MAX_SOLENOIDE        = 8000;  // ms máximo con solenoide activo
const unsigned long PERIODO_MUESTREO       = 10;    // ms (100 Hz)
const float ALFA = 0.2;                 // filtro (0.1 = más suave, 0.5 = más rápido)
const uint8_t MAX_FALLOS = 5;           // lecturas fallidas seguidas permitidas

#define DEBUG 1   // 1 = imprime el ángulo por el monitor serie (115200)

// ---------- Variables de estado ----------
float anguloRef  = 0;        // ángulo con la pierna recta (calibración)
float anguloFilt = 0;        // ángulo relativo filtrado
bool  liberado   = false;    // true = solenoide activo
bool  armado     = true;     // false = debe volver a la extensión antes de liberar de nuevo
unsigned long tCond = 0;     // instante en que empezó la condición actual
unsigned long tLiberado = 0; // instante en que se activó el solenoide
uint8_t fallos = 0;
```

Funciones de lectura, calibración y seguridad

```ino
// ---------- Funciones ----------
void solenoide(bool activo) {
  digitalWrite(PIN_SOLENOIDE, activo ? HIGH : LOW);
}

// Lee el acelerómetro y calcula el ángulo del muslo respecto a la vertical.
// Se asume el eje X del MPU6050 apuntando a lo largo del muslo (hacia abajo).
bool leerAngulo(float &ang) {
  Wire.beginTransmission(MPU_DIR);
  Wire.write(0x3B);                       // registro ACCEL_XOUT_H
  if (Wire.endTransmission(false) != 0) return false;
  if (Wire.requestFrom(MPU_DIR, (uint8_t)6) != 6) return false;

  uint8_t hi, lo;
  hi = Wire.read(); lo = Wire.read(); int16_t ax = (hi << 8) | lo;
  hi = Wire.read(); lo = Wire.read(); int16_t ay = (hi << 8) | lo;
  hi = Wire.read(); lo = Wire.read(); int16_t az = (hi << 8) | lo;

  float yz = sqrt((float)ay * ay + (float)az * az);
  ang = atan2(yz, (float)ax) * 180.0 / PI;  // 0° con el muslo vertical
  return true;
}

// Falla del sensor: solenoide apagado (rodilla bloqueada) y LED parpadeando rápido.
void modoFalla() {
  solenoide(false);
  liberado = false;
  while (true) {
    digitalWrite(PIN_LED, !digitalRead(PIN_LED));
    delay(100);
  }
}

void calibrar() {
  float suma = 0, a;
  for (uint8_t i = 0; i < 50; i++) {
    if (!leerAngulo(a)) modoFalla();
    suma += a;
    if (i % 10 == 0) digitalWrite(PIN_LED, !digitalRead(PIN_LED));
    delay(20);
  }
  anguloRef = suma / 50.0;
```

Inicialización del sistema

```ino
oid setup() {
 pinMode(PIN_SOLENOIDE, OUTPUT);
 solenoide(false);                  // arranca bloqueado
 pinMode(PIN_LED, OUTPUT);
 digitalWrite(PIN_LED, LOW);

 Serial.begin(115200);
 Wire.begin();
if defined(WIRE_HAS_TIMEOUT)
 Wire.setWireTimeout(3000, true);  // evita que se cuelgue si el sensor se desconecta
endif

 // Despertar el MPU6050 (sale del modo sleep)
 Wire.beginTransmission(MPU_DIR);
 Wire.write(0x6B);
 Wire.write(0x00);
 if (Wire.endTransmission() != 0) modoFalla();
 delay(100);

 calibrar();
 anguloFilt = 0;
 digitalWrite(PIN_LED, HIGH);      // LED fijo = listo
```

Funcionamiento del programa principal

```
void loop() {
  static unsigned long tUlt = 0;
  unsigned long ahora = millis();
  if (ahora - tUlt < PERIODO_MUESTREO) return;
  tUlt = ahora;

  // 1) Leer sensor
  float a;
  if (!leerAngulo(a)) {
    if (++fallos >= MAX_FALLOS) {   // sensor perdido: bloquear
      solenoide(false);
      liberado = false;
      tCond = 0;
      digitalWrite(PIN_LED, (ahora / 100) % 2);   // parpadeo = falla
    }
    return;
  }
  fallos = 0;
  digitalWrite(PIN_LED, HIGH);

  // 2) Ángulo relativo y filtrado
  float rel = a - anguloRef;
  anguloFilt = ALFA * rel + (1.0 - ALFA) * anguloFilt;

#if DEBUG
  static unsigned long tPrint = 0;
  if (ahora - tPrint >= 100) {
    tPrint = ahora;
    Serial.print(anguloFilt);
    Serial.print('\t');
    Serial.println(liberado ? 1 : 0);
  }
#endif
```

```
#endif

  // 3) Lógica de liberar / reenganchar
  if (!liberado) {
    if (!armado) {
      // tras un corte por tiempo, esperar a que la pierna vuelva a la extensión
      if (anguloFilt < ANGULO_REENGANCHE) armado = true;
    } else if (anguloFilt > ANGULO_LIBERAR) {
      if (tCond == 0) tCond = ahora;
      if (ahora - tCond >= T_CONFIRMAR_LIBERAR) {
        solenoide(true);
        liberado = true;
        tLiberado = ahora;
        tCond = 0;
      }
    } else {
      tCond = 0;
    }
  } else {
    if (anguloFilt < ANGULO_REENGANCHE) {
      if (tCond == 0) tCond = ahora;
      if (ahora - tCond >= T_CONFIRMAR_REENGANCHE) {
        solenoide(false);
        liberado = false;
        tCond = 0;
      }
    } else {
      tCond = 0;
    }

    // Límite de tiempo del solenoide (protege la bobina y la batería)
    if (liberado && (ahora - tLiberado > T_MAX_SOLENOIDE)) {
      solenoide(false);
      liberado = false;
      armado = false;
      tCond = 0;
    }
  }
}
```

MUCHAS GRACIAS